

# Code Challenge: Mini Inference Router

## Case study

Our products call language models through one shared gateway instead of talking to providers directly. The gateway picks which model backend serves a request, enforces what each tenant is allowed, records what the request cost, and exposes task capabilities that product teams call without knowing which model answered.

Build a small version of that, with one capability on top of it: a customer support assistant that answers a customer message from a support knowledge base and returns the detected intent.

Dataset: [Bitext customer support dataset](#), around 27,000 question and answer pairs across 27 intents. Use a slice as the knowledge base and a held-out slice for evaluation.

## What to build

### Gateway

- Chat endpoint with streaming, authenticated by a per-tenant API key
- Per-tenant quota that fails closed with a clear error
- Metering per request: model used, tokens, latency, estimated cost, outcome, stored in a database

### Routing

- At least two model backends, with rules you can defend for choosing between them
- Fallback when a backend fails or is too slow, with the decision recorded and inspectable

### Support assistant

- Retrieval over the knowledge base slice
- Returns answer, detected intent, retrieved entries, and a confidence signal
- Handles the case where the model returns something unusable, and refuses instead of guessing when confidence is low

### Console (Vue, React, or Next.js, back end Node.js)

- Chat playground where the answer streams in and you can see which model served it, whether fallback fired, what was retrieved, the intent, tokens, latency, and cost
- A simple usage view: requests and cost per tenant, and remaining quota

## Evaluation

- Around 30 held-out cases, with a script that reports intent accuracy, answer quality by a method of your choosing, latency, and cost
- Run it against two configurations, for example two prompts or two models, and compare

## Constraints

- **We do not provide LLM API keys.** You may use a local model (Ollama or similar), a provider free tier, or both. A mock backend with configurable latency and failure is a fine second backend, and the easiest way to show fallback working. At least one real model must serve real requests.
- **Deploy it.** Vercel, Render, or anything similar. Give us a working URL alongside the repository.
- **One day of work.** Cut scope deliberately rather than leaving things half-built, and say what you cut. A smaller system that is measured beats a larger one that is untested.

## Deliverables

- The GitHub repository link (source code and documentation) and a video demo
- A brief technical report explaining your design choices: routing rules, model and retrieval choices, how you evaluated, and the trade-offs you accepted
- The deployed URL

## What we assess

- Correctness of the request path: auth, quota, streaming, failure behaviour
- Routing and fallback: reasoned rather than arbitrary, and observable
- Measurement: tokens, latency, and cost recorded accurately, quality shown with numbers
- Code structure and exception handling: invalid input, timeouts, bad model output
- Judgement: what you built, what you skipped, and whether you said so